ATIVIDADE	PROJETO INTEGRADOR (PWBE)

CONTEXTO e DESCRIÇÃO

A escola **TecnoVille** está desenvolvendo um projeto inspirado no conceito de *Smart City*. Sensores instalados em praças, corredores, pátios e outros espaços enviarão dados de temperatura, umidade, luminosidade e contagem de eventos.

Sua tarefa é desenvolver individualmente o **back-end do api_smart**, utilizando Django, Django REST Framework e MySQL. Os dados serão recebidos e consultados por uma API REST. O painel web poderá ser desenvolvido futuramente e **não faz parte desta entrega**.

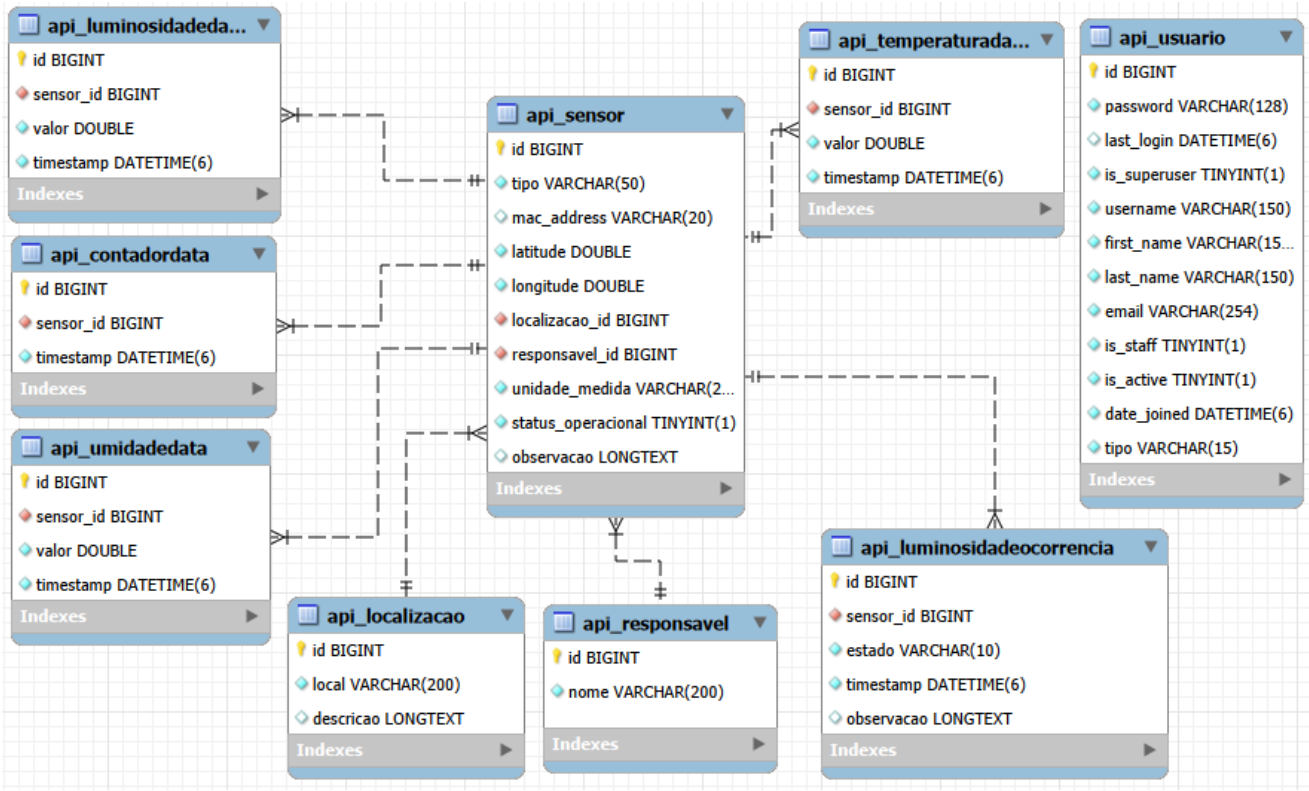


Diagrama Entidade-Relacionamento

Crie os modelos e relacionamentos conforme o DER. Os nomes dos campos, suas características e as relações entre as entidades devem seguir o diagrama fornecido. Os dados iniciais para popular o banco serão disponibilizados pelo professor em planilhas.

Além do que está representado no DER, implemente os seguintes conjuntos de opções com `models.TextChoices`:

Campo	Valor armazenado no BD	Opção exibida
Tipo de usuário	`ADM`	Administrador
Tipo de usuário	`USER`	Usuário
Tipo de sensor	`TMP`	Temperatura
Tipo de sensor	`UMI`	Umidade
Tipo de sensor	`CTD`	Contador
Tipo de sensor	`LUM`	Luminosidade
Unidade de medida	`C`	°C
Unidade de medida	`%`	%
Unidade de medida	`Unidade`	Uni
Unidade de medida	`lux`	lux
Estado da ocorrência de luminosidade	`ALERTA`	Alerta - Luz acesa sem movimento
Estado da ocorrência de luminosidade	`NORMAL`	Normal - Condição normal

Autenticação e permissões

Utilize um modelo de usuário baseado em `AbstractUser` e implemente autenticação com JSON Web Tokens (JWT).

O **token de acesso** deverá ter validade de **25 minutos**. O **token de atualização (*refresh token*)** deverá ter validade máxima de **12 horas**. Após esse período, o usuário precisará realizar um novo login.

Para a avaliação em ambiente de desenvolvimento, crie um superusuário com as credenciais:

- **Username:** `senai`
- **Password:** `123`

Configure as permissões da API de acordo com o tipo de usuário:

- **Administrador:** pode cadastrar, consultar, editar e excluir os dados.
- **Usuário:** pode apenas consultar os dados.

As permissões devem ser verificadas nos endpoints da API, inclusive quando uma requisição for feita diretamente pelo Postman ou Insomnia.

Funcionalidades obrigatórias

Desenvolva endpoints para:

1. Cadastrar, listar, consultar, editar e excluir os registros previstos no DER, respeitando as permissões de cada tipo de usuário.
2. Receber e armazenar os dados enviados pelos sensores.
3. Consultar os dados registrados por um sensor específico.
4. Consultar as leituras mais recentes, incluindo um filtro para as **últimas 24 horas**.
5. Registrar e consultar as ocorrências de luminosidade.
6. Obter e atualizar os tokens JWT.

As respostas de listagem devem permitir que os registros sejam apresentados em formato tabular por um cliente da API. Nesta atividade, a consulta e a conferência dos dados serão feitas pelo **Postman ou Insomnia**; não é necessário criar páginas web.

Importação, simulação e testes

O professor disponibilizará planilhas Excel com dados para popular o banco. O aluno deverá criar um script que leia essas planilhas e importe os registros, respeitando os campos e relacionamentos definidos no DER. A importação deverá ser demonstrada com dados gravados e consultáveis pela API.

O aluno deverá também utilizar uma ferramenta de inteligência artificial para gerar arquivos .sql com dados adicionais para popular o banco MySQL. Cabe ao aluno revisar o SQL produzido, corrigir eventuais erros, verificar a compatibilidade com os modelos e executar o script. O uso de IA não dispensa a compreensão do conteúdo gerado.

Crie, na raiz do repositório, uma pasta chamada DATE. Ela deverá conter todos os arquivos utilizados para popular o banco:

- As planilhas Excel fornecidas pelo professor.
- O script de importação das planilhas.
- Os arquivos .sql gerado com auxílio de IA e revisado pelo aluno.
- O texto do prompt utilizado para gerar o SQL.
- Os scripts e arquivos de dados usados na simulação dos sensores.

O arquivo .sql deverá conter comandos de inserção de dados compatíveis com as tabelas criadas pelas migrações do Django. Os registros devem respeitar a ordem de inserção

necessária às chaves estrangeiras. Não inclua senhas ou credenciais de hospedagem nesses arquivos.

Teste no Postman ou Insomnia o cadastro, a consulta, a edição, a exclusão, os filtros de leitura, a autenticação e as permissões. Demonstre que um usuário comum consegue consultar os dados, mas não consegue alterá-los.

Funcionalidades extras

Implemente **pelo menos cinco** dos recursos abaixo. Eles não substituem as funcionalidades obrigatórias:

- Paginação dos resultados.
- Pesquisa e ordenação.
- Endpoint com resumo estatístico por sensor.
- Alertas para medições fora dos limites definidos.
- Detecção de sensores sem comunicação.
- Validação da compatibilidade entre tipo de sensor e unidade de medida.
- Prevenção de medições duplicadas.
- Testes automatizados da API.
- Documentação automática com Swagger.
- Comando Django para geração de medições simuladas.
- Registro de manutenção dos sensores.
- Auditoria das operações realizadas pelos usuários.

Entrega e avaliação

O trabalho é **individual**. O projeto deverá estar em um **repositório privado**, com acesso concedido ao professor pelo e-mail **lindomarbatistao@gmail.com**.

A data de entrega fica para o dia **30/11/2026**.

Na data de entrega, o professor avaliará diretamente o estado do repositório naquele momento. Alterações posteriores não serão consideradas.

Durante a execução, as dúvidas encaminhadas ao professor deverão tratar exclusivamente da **interpretação do enunciado**. Não haverá orientação técnica detalhada sobre a implementação.

Em caso de suspeita de plágio ou cópia parcial ou total, os trabalhos envolvidos serão analisados; a aplicação de nota zero dependerá da apuração da situação.

Critérios de avaliação

Critérios críticos — 50 pontos

Correspondem ao funcionamento básico da aplicação, demonstrado localmente.

Nº	Critério	Pontos
1	Projeto Django configurado com Django REST Framework, aplicação, URLs e dependências necessárias	4
2	Modelos e campos criados conforme o DER fornecido	7
3	Chaves estrangeiras e relacionamentos do DER funcionando	5
4	Banco local funcionando e permitindo gravar e consultar dados	4
5	Migrações aplicadas e banco local funcionando	4
6	`TextChoices` implementados para tipo de usuário, tipo de sensor, unidade de medida e estado da ocorrência de luminosidade	5
7	Serializers e CRUD básico funcionando para sensores, localizações e responsáveis	8
8	Endpoints para cadastrar e listar dados de temperatura, umidade, contador e luminosidade	5
9	Login por JWT funcionando, com emissão de token de acesso	4
10	Endpoints protegidos contra acesso sem autenticação	4
	Total	**50**

Critérios desejáveis — 50 pontos

Avaliam regras de negócio, operação nos dois serviços de hospedagem e domínio da solução.

Nº	Critério	Pontos
11	CRUD dos demais recursos previstos no DER e consulta de dados por sensor	5
12	Filtros por sensor, período, status e localização, incluindo as últimas 24 horas	6
13	Validações de regras de negócio, incluindo compatibilidade entre sensor e leitura e bloqueio de leituras de sensor inativo	4
14	Token de acesso com validade de 25 minutos, refresh token com validade máxima de 12 horas e renovação funcionando	5
15	Permissões por tipo de usuário: Administrador realiza CRUD; Usuário apenas consulta	5
16	Script que importa corretamente os dados das planilhas Excel fornecidas pelo professor	4
17	Arquivo `.sql` gerado com auxílio de IA, revisado e executado no MySQL, com prompt e arquivos organizados na pasta `DATE`	3
18	API publicada e testada no PythonAnywhere com SQLite	5
19	API publicada e testada no alwaysdata com MySQL/MariaDB e dados importados	5
20	Pelo menos 5 funcionalidades extras implementadas e demonstradas	3
21	Explicação oral dos relacionamentos, autenticação, importações e funcionamento das APIs publicadas	5
	Total	**50**